

EXPERIMENT-9

Student Name: Husanpreet Kaur
Branch: AIT-CSE (Cyber Security)
Semester: 5
Subject Name: Computer Networks

UID: 24BCY70138
Section/Group: 24AIT_KRG-G2
Date: 16/09/2026
Subject Code: 24CSH-337

AIM: To implement a client-server communication system using Python UDP sockets over the IPv4 loopback interface, and to demonstrate connectionless message transmission, server responses, and termination of communication using a predefined message.

OBJECTIVES:

- To understand the basic concept of socket programming.
- To understand the client-server communication model.
- To understand the concept of connectionless communication using UDP.
- To understand the difference between TCP and UDP communication.
- To create an IPv4 UDP socket using Python's socket module.
- To configure a UDP server socket using bind().
- To send data from the client using sendto().
- To receive data at the server using recvfrom().
- To send a response from the server using sendto().
- To receive the server response at the client using recvfrom().
- To understand encoding and decoding of messages during socket communication.
- To use the localhost address 127.0.0.1 and UDP port 5000 for local communication.
- To implement a simple request-response communication system.
- To terminate communication using the predefined message "Over".
- To understand that UDP does not require connection establishment using connect(), listen(), or accept().

SOFTWARE REQUIREMENTS:

(a) Software:

EXPERIMENT-9

- Python 3.x
- Visual Studio Code
- Windows Operating System
- Python socket module (built-in; no external package required)

(b) Hardware (Virtual):

- One computer/laptop
- Network interface with TCP/IP support
- Local loopback interface

THEORY: (i) What is Socket Programming?

Socket programming is a method of enabling two programs to communicate over a network. A socket acts as an endpoint of communication. In a client-server model, the server creates a socket and waits for data, while the client creates a socket and sends data to the server.

In this experiment, UDP sockets are used for communication.

(ii) What is a Socket?

A socket is a software endpoint used by applications to send and receive data over a network.

A socket is associated with:

- IP address
- Port number
- Transport protocol

In this experiment, Python creates an IPv4 UDP socket using:

`socket.AF_INET`

and:

`socket.SOCK_DGRAM`

Socket Parameters:

EXPERIMENT-9

Parameter	Meaning
AF_INET	Uses IPv4 addressing.
SOCK_DGRAM	Creates a UDP datagram socket.
127.0.0.1	IPv4 loopback/localhost address used for communication on the same computer.
5000	UDP port on which the server receives messages.

(iii) What is UDP?

UDP (User Datagram Protocol) is a connectionless transport-layer protocol.

Unlike TCP, UDP does not establish a connection before sending data. It sends independent datagrams from one endpoint to another.

UDP provides:

- Connectionless communication
- Low communication overhead
- Faster transmission
- Datagram-based communication

However, UDP does not inherently guarantee:

- Delivery
- Ordering
- Retransmission
- Duplicate protection

(iv) Client-Server Model:

EXPERIMENT-9

The server starts first and binds its UDP socket to an IP address and port. The client then sends a datagram to the server's IP address and port. There is no connection establishment between the client and server.

Component	Main Responsibility
Server	Creates socket, binds IP/port, receives messages and sends replies.
Client	Creates socket, sends messages and receives replies.

(v) Important UDP Socket Functions:

Function	Purpose
socket()	Creates a new socket.
bind()	Associates the server socket with an IP address and port.
sendto()	Sends data to a specified IP address and port.
recvfrom()	Receives data and the sender's address.
encode()	Converts a string into bytes.
decode()	Converts bytes into a string.
close()	Closes the socket.

(vi) IPv4 Loopback Address:

The address 127.0.0.1 is the IPv4 loopback address. It refers to the same computer on which the program is running. Therefore, this experiment does not require two physical computers. Both the UDP server and UDP client can run on the same computer.

(vii) Port Number:

A port identifies a particular application endpoint on a host. In this experiment, TCP port 5000 is used by the server. The client connects to 127.0.0.1:5000.

EXPERIMENT-9

(viii) Encoding and Decoding:

Socket communication works with bytes.

Before sending a string: `message.encode()`

converts the string into bytes.

For example:

```
message = "Hello Server"
```

```
message.encode()
```

The receiver converts the received bytes back into a string using: `data.decode()`

(ix) UDP Communication and Data Flow:

The communication sequence used in this experiment is:

1. Server creates an IPv4 UDP socket.
2. Server binds the socket to 127.0.0.1 and port 5000.
3. Server waits for a datagram using `recvfrom()`.
4. Client creates an IPv4 UDP socket.
5. Client enters a message.
6. Client sends the message using `sendto()`.
7. Server receives the message and client address using `recvfrom()`.
8. Server displays the message.
9. Server enters a reply.
10. Server sends the reply using `sendto()`.
11. Client receives the reply using `recvfrom()`.
12. Communication continues until the client sends "Over".
13. Both programs close their sockets.

EXPERIMENT-9

STEPS/IMPLEMENTATION:

Part I – Prepare the Project:

1. Open Visual Studio Code.
2. Open the UDP_Socket_Experiment project folder.
3. Ensure that server.py and client.py are present in the project folder.
4. Python 3.x should be installed and available in the VS Code terminal.

Part II – Server Program:

1. Open server.py.
2. Import the socket module.
3. Create an IPv4 UDP socket.
4. Bind the socket to 127.0.0.1 and port 5000.
5. Display the server status.
6. Wait for incoming datagrams using `recvfrom(1024)`.
7. Decode the received bytes.
8. Display the client message.
9. Check whether the received message is "Over".
10. If it is not "Over", enter a reply.
11. Send the reply using `sendto()`.
12. Repeat the process.
13. Close the server socket after receiving "Over".

--- server.py---

```
import socket
```

```
# Creating a UDP Socket
```

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

EXPERIMENT-9

```
# Bind the socket to localhost and port 5000
server_socket.bind(("127.0.0.1", 5000))
print("UDP Server started")
print("Waiting for messages...")
while True:
    message, client_address = server_socket.recvfrom(1024)
    message = message.decode()
    print("Client:", message)
    if message == "Over":
        break
    reply = input("Enter reply: ")
    server_socket.sendto(reply.encode(), client_address)
server_socket.close()
print("Server closed")
```

Part III- Client Program:

1. Open client.py.
2. Import the socket module.
3. Create an IPv4 UDP socket.
4. Define the server address as 127.0.0.1:5000.
5. Enter a message from the user.
6. Encode the message into bytes.
7. Send the message using sendto().
8. If the message is "Over", terminate the loop.

EXPERIMENT-9

9. Otherwise, receive the server response using `recvfrom()`.
10. Decode and display the server response.
11. Repeat the process.
12. Close the client socket.

---client.py---

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

server_address = ("127.0.0.1", 5000)

print("UDP Client started")

while True:

    message = input("Enter message: ")

    client_socket.sendto(message.encode(), server_address)

    if message == "Over":

        break

    reply, server_address_received = client_socket.recvfrom(1024)

    print("Server:", reply.decode())

client_socket.close()

print("Client closed")
```

PROCEDURE:

1. Network Setup:

- Open Visual Studio Code.
- Open the UDP_Socket_Experiment project.
- Verify that `server.py` and `client.py` are available.
- Open the integrated terminal.

EXPERIMENT-9

2. Server Configuration:

- Create an IPv4 UDP socket.
- Bind the socket to 127.0.0.1:5000.
- Wait for incoming datagrams using `recvfrom()`.

3. Client Configuration:

- Create an IPv4 UDP socket.
- Set the server address to 127.0.0.1:5000.
- Send the message using `sendto()`.

4. Data Communication:

- Enter a message on the client.
- Send the message to the server.
- Server receives the message.
- Server displays the message.
- Server enters a reply.
- Server sends the reply to the client.
- Client receives and displays the reply.

5. Connection Termination:

- Enter "Over" on the client.
- The client sends "Over" to the server.
- The server receives "Over".
- The server exits its loop.
- Both sockets are closed.

INPUT/OUTPUT DETAILS:

(a) Communication Parameters:

Parameter	Value
Communication Model	Client-Server
Transport Protocol	UDP

EXPERIMENT-9

Address Family	IPv4
Server IP	127.0.0.1
Server Port	5000
Client	client.py
Server	server.py
Buffer Size	1024 bytes
Communication Type	Connectionless
Socket Type	SOCK_DGRAM
Termination Message	Over

(b) Sample Message Exchange:

Step	Client	Server
1	Starts UDP client	Starts UDP server
2	Sends Hello Server	Receives: Hello Server
3	Waits for response	Sends: Hello Client
4	Displays: Server: Hello Client	Waits for next datagram
5	Sends: How are you?	Receives: How are you?
6	Waits for response	Sends: I am fine
7	Displays Server: I am fine	Waits for next datagram
8	Sends: Over	Receives Over and terminates

COMMANDS: - Create UDP Socket

```
socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

EXPERIMENT-9

- Bind server

```
server_socket.bind(("127.0.0.1", 5000))
```

- Receive UDP datagram

```
message, client_address = server_socket.recvfrom(1024)
```

- Send UDP datagram

```
server_socket.sendto(reply.encode(), client_address)
```

- Client sends message

```
client_socket.sendto(message.encode(), server_address)
```

- Client receives response

```
reply, server_address_received = client_socket.recvfrom(1024)
```

- Close socket

```
client_socket.close()
```

TCP V/S UDP:

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Socket Type	SOCK_STREAM	SOCK_DGRAM
listen()	Required for server	Not required
accept()	Required	Not required
connect()	Used by client	Not required here
Sending	send()	sendto()
Receiving	recv()	recvfrom()
Reliability	Reliable	No delivery guarantee

EXPERIMENT-9

Ordering	Ordered	No ordering guarantee
Overhead	Higher	Lower
Data Unit	Byte stream	Datagram

DEMONSTRATION/OUTPUT:

```

Problems  Output  Debug Console  Terminal  Ports  Query Resu
▼ Terminal

● PS C:\Users\husan\Downloads\UNIT-2-Chapter 2.2 - CN.pptx
(1)> cd UDP_Socket_Experiment
● PS C:\Users\husan\Downloads\UNIT-2-Chapter 2.2 - CN.pptx
(1)\UDP_Socket_Experiment> cd UDP_Socket_Experiment
○ PS C:\Users\husan\Downloads\UNIT-2-Chapter 2.2 - CN.pptx
(1)\UDP_Socket_Experiment\UDP_Socket_Experiment> python
server.py
UDP Server started
Waiting for messages...
Client: hello
Enter reply: hi

```

```

python3.13 - UDP_Socket_Experiment  +  ▼  □
○ PS C:\Users\husan\Downloads\UNIT-2-Chapter 2.2
- CN.pptx (1)\UDP_Socket_Experiment\UDP_Socket_
Experiment> python client.py
UDP Client started
Enter message: hello
Server: hi
Enter message:

```

EXPERIMENT-9

```
Help  ← →  UNIT-2-Chapter 2.2 - CN.pptx (1)  [Icons]

client.py X
UDP_Socket_Experiment > UDP_Socket_Experiment > client.py > ...

1  import socket
2
3  client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4
5  server_address = ("127.0.0.1", 5000)
6
7  print("UDP Client started")
8
9  while True:
10     message = input("Enter message: ")
11
12     client_socket.sendto(message.encode(), server_address)
13
14     if message == "Over":
15         break
16
17     reply, server_address_received = client_socket.recvfrom(1024)
18
19     print("Server:", reply.decode())
20
21 client_socket.close()
22
23 print("Client closed")
24
```

```
Help  ← →  UNIT-2-Chapter 2.2 - CN.pptx (1)  [Icons]

server.py X
UDP_Socket_Experiment > UDP_Socket_Experiment > server.py > ...

1  import socket
2
3  ## Creating a UDP Socket
4  server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
5
6  ## A socket is a software endpoint through which one program sends or receives data.
7
8  server_socket.bind(("127.0.0.1", 5000))
9
10 print("UDP Server started")
11 print("Waiting for messages...")
12
13 while True:
14
15     message, client_address = server_socket.recvfrom(1024)
16
17     message = message.decode()
18
19     print("Client:", message)
20
21     if message == "Over":
22         break
23
24     reply = input("Enter reply: ")
25
26     server_socket.sendto(reply.encode(), client_address)
27
28 server_socket.close()
29
30 print("Server closed")
```

EXPERIMENT-9

RESULT: The experiment was successfully implemented using Python UDP socket programming. An IPv4 UDP server was created and bound to 127.0.0.1:5000, while the client sent datagrams to the server using `sendto()`.

The server received client messages using `recvfrom()` and sent responses back to the client using `sendto()`. String encoding and decoding were used for byte-based socket communication.

The experiment demonstrated connectionless communication using UDP, without using `listen()`, `accept()`, or a TCP-style connection establishment process. The communication was terminated using the "Over" message, followed by proper socket closure.

LEARNING OUTCOMES:

After completing this experiment:

- I understood the client-server model and the role of UDP sockets in network communication.
- I learned to create and configure IPv4 UDP sockets using Python's socket module.
- I learned the use of `bind()`, `sendto()`, `recvfrom()`, and `close()` for UDP communication.
- I understood the difference between connection-oriented TCP and connectionless UDP communication.
- I developed practical skills in implementing and testing UDP client-server communication using localhost and a UDP port.